Multi-threading

Alessandro Vulcu

December 1, 2025

Contents

1 The Imperative for Parallel Programming

In modern computing, the era of relying solely on increasing processor clock speeds for performance gains has ended. Instead, the industry has shifted towards **multi-core architectures**, making parallel programming an essential paradigm, not an esoteric specialty. The fundamental principle of sequential execution—performing one instruction at a time—is no longer sufficient for building responsive, efficient, and powerful software. **Multi-threading** and **multi-processing** are the core techniques that allow a program to perform two or more operations concurrently, unlocking the full potential of today's hardware.

A clear, practical scenario where parallel operations are non-negotiable is a bidirectional chat application. Such an application must simultaneously listen for incoming messages from a network socket and read user input from the keyboard. A sequential approach would be unworkable; the program would be stuck waiting for either user input or a network message, unable to do both at once. The only viable solution is to handle these two tasks in parallel.

This is achieved by dedicating a separate thread to each task:

```
// Thread 1: read from standard input and send to socket
while(true) {
    getline (std::cin, data);
    write(sk_fd,data,data.size());
}

// Thread 2: read from socket and print to standard output
while(true) {
    read(sk_fd, rx_data, MAX_SIZE);
    std::cout << rx_data << std::endl;
}
```

In this model, Thread 1 blocks only when waiting for the user to type, while Thread 2 blocks only when waiting for data from the network. Neither thread's blocking state prevents the other from executing. This **separation of concerns** is the essence of concurrent design. To understand how to build such systems, we must first examine the foundational models provided by the operating system that enable this parallelism.

2 Foundational Models of Concurrency

To effectively implement parallelism, a developer must first understand the core abstractions that operating systems provide to manage concurrent execution. These foundational models are the **process** and the **thread**. They offer different trade-offs and form the building blocks of any parallel program.

2.1 The Process Model

A process is the operating system's abstraction of a running program. On a single-CPU system, true parallel execution is an illusion; the OS kernel rapidly switches the CPU between different programs, creating the appearance of simultaneous activity. To manage this complexity, designers developed the conceptual model of **sequential processes**.

The state of a process encapsulates everything required for its execution. This includes:

- **The Program:** The executable code being run.
- **CPU State:** The current values of the program counter, processor registers, and other CPU-specific data.
- **Memory Address Space:** The process's dedicated memory, containing variable values and other runtime data.
- **Resource State:** Information about other resources in use, such as open files or I/O devices.

This collection of state is precisely the information the operating system must save and restore when it switches the CPU from one process to another. This operation, known as a **Context Switch**, is the

fundamental mechanism that enables multi-processing. It is critical to distinguish between a static program and a dynamic process. A program is merely a passive entity—an algorithm expressed in code. A process is the active execution of that program, characterized by its unique input, output, and state. Multiple distinct processes can execute the same program, each maintaining its own independent state.

2.2 Process State Transitions

A process moves between several states during its lifecycle, managed by the OS scheduler. The three core states are:

- **Running:** The process is actively using the CPU at that instant.
- **Ready:** The process is runnable and waiting for its turn on the CPU.
- **Blocked:** The process cannot run because it is waiting for an external event, such as I/O completion or resource acquisition.

The transitions between these states follow a well-defined pattern, as summarized below:

[Image of Process State Transition Diagram]

From State	To State	Transition Trigger	Transition Type
Running	Blocked	Process needs to wait for a resource (e.g., input)	Voluntary
Blocked	Ready	The awaited resource has become available	Involuntary
Ready	Running	The OS scheduler allocates the CPU to the process	Involuntary
Running	Ready	The scheduler preempts the process for another	Involuntary

While processes provide robust isolation, a more lightweight model for concurrency, the thread, operates within the process environment itself.

3 Threads vs. Processes

The choice between a multi-threading or multi-processing architecture is a critical design decision with significant implications for performance, complexity, and robustness. A **thread** can be defined as the system-level facility for executing a task. What distinguishes threads from processes is their ability to allow multiple, largely independent executions to take place within the **same process environment**. This shared environment is the source of both their primary advantages and disadvantages.

Advantages of Threads	Disadvantages of Threads
Shared Address Space: All threads within a process share the same memory, which vastly simplifies communication and data sharing between them compared to inter-process communication (IPC).	**Single Point of Failure:** If a single thread encounters a critical error and crashes, it brings down the entire process. In contrast, a multi-process architecture offers greater resilience; if one process crashes, others can continue operating and the failed process can be restarted independently.
Lower Overhead: Threads are more "lightweight" than processes. They are faster to create, destroy, and manage because they do not require duplicating the entire process context.	**No Distributed Execution:** A multi-threaded program is confined to a single machine, as all threads must share the same process address space. A multi-process program, however, can be distributed, with different processes running on different machines across a network.
Efficient Context Switching: Switching between threads within the same process is significantly faster than switching between processes, as the memory address space does not need to be reloaded.	

For the scope of this report, the focus will be on the multi-threading model, which is exceptionally common for applications that do not require distributed execution.

4 Implementing and Managing Threads in C++11

The **C++11 standard** marked a revolution for concurrent programming in the language. It introduced a standard library for thread management (`<thread>`) that abstracts away platform-specific APIs (like POSIX threads), providing a portable and modern interface for creating and managing threads.

4.1 The `std::thread` Class and Compilation

The primary mechanism for creating a new thread is the `std::thread` class. Its constructor takes the task to be executed and any arguments that task requires. The constructor signature is: `thread(Function&& f, Args&& ... args);`

- `f` is a callable object representing the task for the new thread to execute. This can be a function pointer, a lambda expression, or any object that can be invoked like a function.
- `args` is a variable-length list of arguments to be passed to the task `f`.

If the system cannot start a new thread for any reason (e.g., resource limits), the constructor will throw a `std::system_error`. Because the C++ standard library's thread implementation often relies on the underlying operating system's native threading library (like POSIX threads or pthreads on Linux-based systems), it is necessary to link against it during compilation. This is commonly managed in build systems like Autotools with a configuration check:

```
AC_CHECK_LIB(pthread, pthread_create, [LIBS="$LIBS -lpthread"])
```

This command checks for the existence of the `pthread_create` function within the `pthread` library. If found, it adds the `-lpthread` linker flag to the build configuration, ensuring the program compiles and links correctly.

4.2 Critical Lifecycle Operations: `join()` vs. `detach()`

A fundamental rule of `std::thread` management is that a thread object must be either **joined** or **detached** before its destructor is called (i.e., before it goes out of scope). Failure to do so results in `std::terminate` being called, abruptly ending the program.

- `join()`: This operation blocks the calling thread (e.g., main) and makes it wait until the spawned thread has completed its execution. It is the standard mechanism for synchronizing the completion of a task.
- `detach()`: This operation separates the thread of execution from the `std::thread` object. The thread is allowed to run independently in the background, becoming a "daemon" thread. The `std::thread` object no longer has any control over it. This approach is described as "really dangerous" and should generally be avoided. A detached thread can easily outlive the scope of variables it references, leading to **undefined behavior**.

It is an error to call `join()` on a thread that has already been detached. To safely manage threads, the `joinable()` method can be used to check if a thread object is associated with an active thread of execution before attempting to join it.

```
if(thr.joinable()) {
    thr.join();
}
```

4.3 Thread Identification and Safe Management

Every active thread has a unique identifier of type `std::thread::id`. This ID can be retrieved in two ways:

- `std::this_thread::get_id()`: Returns the ID of the thread from which it is called.

- `thr.get_id()`: Returns the ID of the thread managed by the `std::thread` object `thr`. This can only be called on joinable threads.

Calling `get_id()` on a `std::thread` object that does not represent an active thread of execution (e.g., after it has been detached or joined) returns a default-constructed ID, signifying "not any thread".

```
#include <iostream>
#include <thread>

void simple_task() { /* do something */ }

int main() {
    std::thread thr(simple_task);
    std::cout << "Thread ID before detach: " << thr.get_id() << std::endl;
    thr.detach();
    // After detach, the thread object is no longer associated with
    // the thread of execution.
    std::cout << "Thread ID after detach: " << thr.get_id() << std::endl;
    return 0;
}
```

A common challenge in thread management is ensuring `join()` is called even if a function terminates prematurely, for instance, by throwing an exception. If an exception is thrown after a thread is created but before it is joined, the `std::thread` object's destructor will be invoked as the stack unwinds, causing the program to crash. A simple try-catch block is one solution, but it is not considered elegant. The preferred C++ approach is the **Resource Acquisition Is Initialization (RAII)** pattern. By wrapping the `std::thread` object in a class whose destructor guarantees `join()` is called, we can ensure safe cleanup regardless of how the function exits.

```
// RAII wrapper for std::thread
class A {
    std::thread thr;
    void incr() { /* ... */ }

public:
    // Constructor starts the thread
    A() : thr(&A::incr, this) {}

    // Destructor guarantees the thread is joined
    ~A() {
        if (thr.joinable()) {
            thr.join();
        }
    }
};
```

Having covered the mechanics of creating and managing threads, we now turn to the specifics of defining tasks and passing data to them.

5 Defining Tasks and Passing Data to C++ Threads

The flexibility of `std::thread` allows it to execute various types of callable objects. However, understanding its parameter-passing semantics is absolutely crucial for ensuring program correctness and avoiding subtle bugs related to data ownership and lifetime.

5.1 Task Types

The task (`f`) passed to a `std::thread` constructor can be defined in several ways:

- **Standalone Function:** A regular, free function.
- **Class Member Function:** A method of a class. When passing a member function, you must also provide a pointer to the object instance on which the method should be called as the first argument after the function pointer.

- **Lambda Function:** An anonymous, inline function, which is often the most concise and convenient method.

5.2 Parameter Passing Semantics

A critical rule governs how arguments are passed to a new thread: by default, **all arguments are copied or moved into internal storage**. This is a critical, non-intuitive departure from standard function call semantics and a frequent source of bugs for developers new to `std::thread`. The constructor's behavior is to copy/move arguments to ensure their lifetime extends until the thread begins execution. Simply passing a variable, even to a function expecting a reference, will result in a copy.

- **Passing by Reference:** To modify an original variable from within a thread, you must explicitly wrap the argument in `std::ref()` to enforce reference passing.

```
void incr(int& v) {
    ++v;
}

int main() {
    int v = 1;
    std::thread thr(incr, std::ref(v)); // Enforce pass-by-reference
    thr.join();
    // After join(), v will be 2
}
```

Warning: Sharing memory by passing references is inherently dangerous. It introduces the risk of **data races** if multiple threads access the same data without proper synchronization mechanisms (like mutexes).

- **Passing with Move Semantics:** To transfer ownership of data to a thread efficiently (avoiding a copy) without sharing memory, you can use **move semantics** with `std::move()`. This casts the variable to an rvalue reference, allowing its resources to be "moved" into the thread's context. The original variable is left in a valid but unspecified state.

```
void print(const std::string& s) {
    std::cout << "s=" << s << std::endl;
}

int main() {
    std::string v = "Hi!";
    std::thread thr(print, std::move(v)); // Move 'v' into the thread
    std::cout << "v=" << v << std::endl;
    thr.join();
}
// Console Output:
// v=
// s=Hi!
```

As seen in the output, the original variable `v` loses its content, which has been efficiently transferred to the new thread.

6 Leveraging Lambda Functions in Multi-Threading

Lambda functions, introduced in modern C++, provide a highly concise and powerful syntax for creating anonymous function objects. Their ability to be defined inline where they are needed makes them exceptionally well-suited for defining tasks for `std::thread`, improving code readability and locality.

6.1 Anatomy of a Lambda Function

The general syntax of a lambda function is: `[capture](params) → return { body }` Each component serves a specific purpose:

- [ ] (**Capture Clause**): Specifies which variables from the enclosing scope are made available inside the lambda's body and how they are "captured" (by value or by reference).
- ( ) (**Parameter List**): The list of parameters the lambda function accepts, just like a normal function.
- → (**Return Type**): An optional trailing return type. The compiler can often deduce this automatically.
- { } (**Function Body**): The code to be executed when the lambda is called.

6.2 The Capture Clause Explained

The capture clause is the most unique and powerful feature of lambdas. It controls the lambda's access to its surrounding environment.

- `[&a]`: Captures the variable `a` by reference. Changes to `a` inside the lambda will affect the original `a`.
- `[a]`: Captures the variable `a` by value. The lambda gets its own copy of `a`, and changes do not affect the original.
- `[=]`: Default captures all automatic variables used in the lambda's body by value.
- `[&]`: Default captures all automatic variables used in the lambda's body by reference.
- `[this]`: Captures the `this` pointer of the enclosing class by reference, making member variables accessible.

By default, variables captured by value are `const` inside the lambda. To modify a value-captured variable, the lambda must be marked with the `mutable` keyword.

6.3 Lambdas in Action with Threads

The interaction between lambda captures and thread execution is a critical concept to master. Consider the following scenarios, each aiming to increment a variable `a` from a new thread:

```
#include <iostream>
#include <thread>

int main() {
    // Case 1: Capture by reference
    {
        int a = 0;
        std::thread thr([&a]() {
            a = a + 1;
        });
        thr.join();
        // After join, the original 'a' is modified.
        std::cout << "Case 1 (capture by reference): a = " << a << std::endl;
    }

    // Case 2: Capture by value (non-mutable)
    {
        int a = 0;
        std::thread thr([a]() {
            // a = a + 1; // This would cause a COMPILE ERROR,
            // as captured 'a' is const.
        });
        thr.join();
        // The original 'a' is unaffected.
        std::cout << "Case 2 (capture by value): a = " << a << std::endl;
    }

    // Case 3: Capture by value and mutable
    {
```

```
        int a = 0;
        std::thread thr([a]() mutable {
            a = a + 1; // Modifies the lambda's internal copy of 'a'.
        });
        thr.join();
        // The original 'a' is unaffected.
        std::cout << "Case 3 (capture by value, mutable): a = " << a << std::endl;
    }

    // Case 4: Pass by value as a parameter
    {
        int a = 0;
        std::thread thr([](int v) {
            v = v + 1; // Modifies the parameter 'v', which is a copy of 'a'.
        }, a);
        thr.join();
        // The original 'a' is unaffected.
        std::cout << "Case 4 (pass as parameter): a = " << a << std::endl;
    }

    return 0;
}
```

Analysis:

- **Case 1:** `[&a]` captures **a** by **reference**. The thread operates directly on the original **a** in main's scope. Therefore, after the thread joins, **a** is **1**.
- **Case 2:** `[a]` captures **a** by **value**. The lambda receives a `const` copy of **a**. Attempting to modify it would result in a compilation error. The original **a** is never touched and remains **0**.
- **Case 3:** `[a]() mutable` captures **a** by **value** but allows the lambda to modify its internal copy. The thread increments its private copy of **a**, but this has no effect on the original **a**. It remains **0**.
- **Case 4:** The variable **a** is passed as a regular argument to the thread's task. As established, arguments are passed by **value**. The lambda receives a copy (**v**), modifies it, and the original **a** is unaffected, remaining **0**.